

Codeingo

A Field Project Report

BACHELOR OF SCIENCE (COMPUTER SCIENCE)

By

Aryan Sawant

Under the esteemed guidance

of

Ms. Pranali Patil



**DEPARTMENT OF INFORMATION TECHNOLOGY
& COMPUTER SCIENCE**

**KERALALEEYA SAMAJAM (REGD.) DOMBIVLI'S
MODEL COLLEGE**

(EMPOWERED AUTONOMOUS)

DOMBIVLI, 421201

MAHARASHTRA

OCTOBER 2026

KERALEEYA SAMAJAM (REGD.) DOMBIVLI'S MODEL COLLEGE
(EMPOWERED AUTONOMOUS)
(Affiliated to University of Mumbai)
DOMBIVLI- MAHARASHTRA-421201

**DEPARTMENT OF INFORMATION TECHNOLOGY
& COMPUTER SCIENCE**



CERTIFICATE

This is to certify that the project entitled, “**Codeingo**”, is bonafied work of Aryan Sawant bearing Seat No: **[SEAT NO e.g. BSCS/III-2627-XXXX]** submitted in partial fulfilment of the requirements for the award of degree of **BACHELOR OF SCIENCE in COMPUTER SCIENCE** from Keralaleeya Samajam (Regd.) Dombivli’s Model College.

Internal Guide

Coordinator

External Examiner

Date

College Seal

ABSTRACT

Codeingo is a web-based, game-like platform that teaches programming through short, interactive lessons, in the spirit of language-learning apps such as Duolingo. It is designed to address common problems that beginners face when learning to code: confusing syntax, a lack of guidance, no real-world application and a loss of motivation after a few days.

The platform offers ten courses (Python, JavaScript, TypeScript, Java, C++, C, SQL, HTML & CSS, Git, and Data Structures & Algorithms) as a structured path of units and lessons. Learners write and run real programs inside the browser, earn XP, keep daily streaks, collect badges and complete projects at the end of each section. Answers are checked on the server so that expected results cannot be copied from the page. Sign-in uses a Google account protected by mandatory 2-step verification.

The system is built with Next.js and React for the interface and Python FastAPI with a SQL database for the server. Its features were chosen from a requirement survey of 53 young learners, which showed that 62% find a game-like way of learning appealing, that lack of guidance and confusing syntax are the main obstacles, and that building real projects is the strongest motivator. Overall, Codeingo gives beginners one simple, engaging place to learn programming step by step.

Acknowledgement

It gives me a pleasure to present my field project on “**Codeingo**”. This is my milestone in Bachelor of Science (Computer Science). I would like to express my sincere thanks to all the teachers who helped me throughout the project. I would like to acknowledge the help and guidance provided by our guide Pranali Patil in all places during the presentation of this project.

I am grateful to IT-CS Co-Ordinator Dr. Divya Premchandran for her constant support and motivation. I am thankful to our honourable Principal Dr. CA Ravindra P. Bambardekar. Onwards my project works, I am also thankful to the staff members of the IT-CS department for their moral support. I also thank the 53 learners who answered the requirement survey.

Declaration

I hereby declare that the project entitled, “**Codeingo**” done at Keraleeya Samajam (Regd.) Dombivli’s Model College (Empowered Autonomous), has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.

The project is done in partial fulfilment of the requirements for the award of degree of **BACHELOR OF SCIENCE (COMPUTER SCIENCE)** to be submitted as a Third semester field project as part of our curriculum.

TABLE OF CONTENTS

Sr.No	Title	Page No
Chapter 1	Introduction	10
1.1.	Background	10
1.2.	Objectives	10
1.3.	Purpose, Scope & Applicability	10
1.3.1.	Purpose	10
1.3.2.	Scope	11
1.3.3.	Applicability	11
Chapter 2	Survey of Technologies	13
2.1.	Existing System	13
2.2	List of Technologies	13
2.2.1	Next.js	13
2.2.2	React	13
2.2.3	Tailwind CSS	14
2.2.4	Python	14
2.2.5	FastAPI	14
2.2.6	SQLAlchemy	14
2.2.7	PostgreSQL and SQLite	14
2.2.8	JWT cookies and 2-step verification	14
2.2.9	Pyodide and Web Workers	14
2.2.10	Progressive Web App	14
2.3	Comparative Study of Technologies	14
2.4	Selected Technology	15
Chapter 3	Requirements and Analysis	18
3.1.	Problem Definition	18
3.2.	Requirement Specification	18
3.3.	Software & Hardware Requirements	18
3.4	Conceptual Model	19
Chapter 4	Data Analysis, Interpretation and Presentation	21
4.1	Survey Form	21
4.2	Responses	22
4.3	Data Analysis	23
4.4	Conclusion	28

LIST OF TABLES

Sr.no	Name of the table	Page No
2.3.1	Comparative Study	14

LIST OF FIGURES

Sr.no	Name of the Figure	Page No
3.4.1	Conceptual Model of Codeingo	19
4.2.1	Responses	23
4.3.1	Age group of the respondents (Question 2)	24
4.3.2	Earlier programming experience (Question 3)	24
4.3.3	What made learning difficult or boring (Question 4)	25
4.3.4	Preferred way of learning (Question 5)	25
4.3.5	Coding platforms used before (Question 6)	26
4.3.6	Appeal of game-like learning (Question 7)	26
4.3.7	First language to learn (Question 8)	27
4.3.8	What keeps learners motivated (Question 9)	27
4.3.9	Solo or community learning (Question 10)	28
4.3.10	Biggest reason to stop using a platform (Question 11)	28

Periodic Field Project Report

Name of the Student	Aryan Sawant
Program /Semester	III
Roll No/Seat No	[ROLL / SEAT NO e.g. BSCS/III-2627-XXXX]
Field project Title	Codeingo
Name of the Faculty mentor	Ms. Pranali Patil
Overview (Max 150 Words) (Brief summary of key activities, tasks and projects undertaken during the period)	
This period was spent defining the Codeingo project: why a new coding-learning platform is needed, what it should achieve, who will use it and what is inside and outside its scope. Existing apps and the problems beginners face were studied, and a requirement survey was prepared. The main aim is to provide a simple, game-like and secure place where a beginner can learn programming step by step, write real code in the browser and stay motivated through streaks, points and projects.	
Learning Outcomes (Max 100 words): (Highlight the main lessons, skills, or knowledge gained during the week.)	
Through this chapter I learned how to state the background of a problem, write measurable objectives, and separate purpose, scope and applicability. I understood that a project should clearly list its limitations, and that beginners need guidance, feedback and a sense of progress more than long theory.	
Mentor Feedback and Suggestions (Max 100 words): (Feedback received from the mentor and any additional suggestions for improvement.)	
[Feedback from the guide, Ms. Pranali Patil, to be written after the chapter review]	

Student Sign with date

Mentor's Signature with date

Chapter 1 Introduction

1.1. Background

Almost every industry today depends on software, and the ability to write code has become a basic skill. Yet many beginners struggle to start. Installing tools, understanding confusing syntax and having nobody to guide them makes learning slow and boring, and many give up within the first weeks. Traditional ways of learning, such as long videos or text-only tutorials, lack interactive and engaging elements that keep learners motivated.

Language-learning apps such as Duolingo showed that short daily lessons, instant feedback, points and streaks can turn a difficult subject into a daily habit. Recent web technology also makes it possible to run real programming languages inside the browser, so nothing needs to be installed.

The concept of Codeingo is inspired by these ideas. It is a web-based platform in which a learner follows a guided path of courses, units and lessons, writes and runs real programs in the browser, and earns XP, streaks and badges. It builds upon existing work in online coding education and game-based learning to create a more engaging and beginner-friendly solution.

1.2. Objectives

The main objectives of the Codeingo project are:

- To develop an interactive learning platform with a structured path of courses, units and lessons for beginners.
- To let learners write and run real code inside the browser without installing anything.
- To add game elements such as XP, daily streaks, hearts, badges and weekly leagues to improve motivation.
- To include practice, chapter tests and projects so that learners can apply what they learn.
- To protect user accounts with secure sign-in and mandatory 2-step verification.
- To design a simple, user-friendly interface that works on computers and phones in a light and a dark theme.

1.3. Purpose, Scope & Applicability

1.3.1. Purpose

The purpose of this project is to create a digital environment that makes learning programming easy, engaging and habit-forming. The project answers key questions such as:

- How can beginners be guided step by step so that they do not give up?
- How can game-like features and real projects be used to keep learners motivated?

The system helps learners practise regularly, get instant feedback and see their own progress. It also demonstrates how user-interface design, security and learning content can be combined in one application.

1.3.2. Scope

The scope of the Codeingo project includes:

1. User Registration and Login: Users sign in with a Google account and finish 2-step verification (an emailed code, an authenticator app or a passkey).
2. Learning Content: Ten courses with units, lessons and exercises, where learners write and run programs in the browser.
3. Game Elements: XP, streaks, hearts, badges, a daily challenge, chapter tests and weekly leagues.
4. Administration: A hidden admin area where only authorised administrators manage content and users.

Limitations:

- Internet Dependency: The learner's progress is stored on the server, so sign-in and saving progress need an internet connection.
- Languages that need a compiler: Java, C and C++ programs cannot run inside the browser and need a server-side sandbox for full support.
- No video lessons: The lessons are text and exercise based; video explanations are not part of this version.

1.3.3. Applicability

This project is useful for:

- School and college students – who want to start programming without installing tools.
- Self-learners and career-switchers – who need a guided path from beginner to advanced level.
- Teachers and colleges – for contests and practice sessions with a leaderboard.

The system provides both direct and indirect benefits by improving practice habits, reducing the fear of coding and making learning enjoyable. It contributes to the computer field by demonstrating how game-based learning, in-browser code execution and secure sign-in can be combined in one application.

Periodic Field Project Report

Name of the Student	Aryan Sawant
Program /Semester	III
Roll No/Seat No	[ROLL / SEAT NO e.g. BSCS/III-2627-XXXX]
Field project Title	Codeingo
Name of the Faculty mentor	Ms. Pranali Patil
Overview (Max 150 Words) (Brief summary of key activities, tasks and projects undertaken during the period)	
During this period the existing coding-learning systems were studied to understand their features and limitations. The technologies needed for Codeingo were listed and described: Next.js, React, Tailwind CSS, Python, FastAPI, SQLAlchemy, PostgreSQL and SQLite, JWT cookies with 2-step verification, Pyodide with Web Workers, and Progressive Web Apps. A comparative study was made between the selected technologies and their alternatives, and the role of each part of the system was understood.	
Learning Outcomes (Max 100 words): (Highlight the main lessons, skills, or knowledge gained during the week.)	
I learned about the technologies used in building a web-based learning application: how a front end, a REST API and a database work together, how code can be run safely in the browser, and how secure sign-in works. I also learned how to compare technologies and justify the final selection.	
Mentor Feedback and Suggestions (Max 100 words): (Feedback received from the mentor and any additional suggestions for improvement.)	
[Feedback from the guide, Ms. Pranali Patil, to be written after the chapter review]	

Student Sign with date

Mentor's Signature with date

Chapter 2 Survey of Technologies

2.1. Existing System

To understand the requirements of Codeingo, several popular learning platforms and tools were surveyed. The survey respondents also named many of them:

- Codecademy and freeCodeCamp: Interactive course sites with in-browser editors. Codecademy locks many projects and certificates behind a paid plan; freeCodeCamp is free but is mostly text based and gives little game-like motivation.
- Scratch: A visual block-based tool that is excellent for children, but it does not teach real text syntax, so the next step to Python or Java is a big jump.
- LeetCode and video channels (for example Takeuforward, CodeWithHarry): Strong for practice and explanation, but they assume that the learner already knows the basics and do not give a guided path or progress tracking.
- Local compilers and editors (Turbo C, Sublime Text): Need installation and set-up, and give no feedback or guidance.

Out of the 53 survey respondents, 36 had not used any such platform. This shows that a simple, no-installation and guided platform can reach many beginners.

2.2 List of Technologies

The following technologies are used for developing Codeingo:

1. Next.js
2. React
3. Tailwind CSS
4. Python
5. FastAPI
6. SQLAlchemy
7. PostgreSQL and SQLite
8. JWT cookies and 2-step verification
9. Pyodide and Web Workers
10. Progressive Web App

2.2.1 Next.js

Next.js is a React framework used to build the web pages and the installable app of Codeingo. It provides routing, server rendering and a production build, and lets the same code serve the landing page, lessons and admin area.

2.2.2 React

React is a JavaScript library for building user interfaces from small reusable components. In Codeingo it is used for the lesson screens, quizzes, the learning path and the animated mascot.

2.2.3 Tailwind CSS

Tailwind CSS is a utility-first styling tool. Colours are defined as variables, which allows the light theme (white and blue) and the dark theme (black and green) to be switched with one setting.

2.2.4 Python

Python is the programming language of the server. It is simple to read, has a large library collection and is also the first language most survey respondents want to learn.

2.2.5 FastAPI

FastAPI is a Python framework for building REST APIs. It validates incoming data automatically, is fast, and handles sign-in, lessons, progress, practice and contests.

2.2.6 SQLAlchemy

SQLAlchemy is an object-relational mapper. It lets the server work with database tables through Python classes such as users, courses, lessons and attempts.

2.2.7 PostgreSQL and SQLite

PostgreSQL is the production database that stores users, lessons, progress and certificates. SQLite is a light file database used during development and testing, so the same code runs on both.

2.2.8 JWT cookies and 2-step verification

After sign-in the server issues short-lived signed tokens (JSON Web Tokens) in secure cookies. Sign-in needs a second step: an emailed code, an authenticator app or a passkey (WebAuthn).

2.2.9 Pyodide and Web Workers

Pyodide runs Python inside the browser using WebAssembly, and Web Workers run JavaScript, TypeScript and SQL in the background. This lets learners run real code safely without installing anything.

2.2.10 Progressive Web App

A progressive web app can be installed from the browser to the home screen and opens like a normal app, with its own icon and an offline page.

2.3 Comparative Study of Technologies

The following table compares the selected technologies with related technologies that could also be used for developing similar functionality.

Table 2.3.1: Comparative Study

No	Selected Technology	Related Technology	Comparison	Reason for Selecting
1	Next.js	Create React App / plain React	Next.js adds routing, server rendering and an optimised build. Plain React needs these to be added separately.	Next.js is selected because it gives pages, routing and a production build in one framework.

No	Selected Technology	Related Technology	Comparison	Reason for Selecting
2	React	Angular	React is a light library with a large community. Angular is a complete framework with a steeper learning curve.	React is selected because it is easier to learn and fits small reusable lesson components.
3	Tailwind CSS	Bootstrap	Tailwind provides low-level utilities and full control of design. Bootstrap provides ready-made components with a fixed look.	Tailwind is selected because Codeingo needs its own look and two colour themes.
4	Python	Java	Python has simple syntax and short code. Java is more verbose and needs compilation.	Python is selected because it is fast to develop in and is the most requested first language in the survey.
5	FastAPI	Django / Flask	FastAPI validates data and creates API documentation automatically. Django is larger; Flask has fewer built-in features.	FastAPI is selected because the project mainly needs a fast, typed REST API.
6	SQLAlchemy	Raw SQL queries	SQLAlchemy maps tables to Python classes and protects against SQL injection. Raw SQL is harder to maintain.	SQLAlchemy is selected because it keeps the code clean and works with several databases.
7	PostgreSQL / SQLite	MongoDB	SQL databases use tables with relations and strict rules. MongoDB stores flexible documents.	A relational database is selected because users, lessons and progress are closely related.
8	JWT cookies + 2-step verification	Server sessions / passwords only	Signed tokens are stateless and easy to scale; passwords alone are weaker.	Selected because it is secure, and the survey groups are young learners who should not reuse passwords.
9	Pyodide / Web Workers	Running code on the server	In-browser execution costs the server nothing and is isolated; server execution is risky and expensive.	Selected because it is safe and needs no installation.
10	Progressive Web App	Native Android app	A PWA reuses the web code and installs from the browser. A native app needs separate code per platform.	Selected because it reaches phones and computers with one code base.

2.4 Selected Technology

The technologies selected for Codeingo are Next.js, React, Tailwind CSS, Python, FastAPI, SQLAlchemy, PostgreSQL with SQLite, JWT cookies with 2-step verification, Pyodide with Web Workers, and a Progressive Web App. The front end is developed using Next.js, React and Tailwind CSS: React builds the screens, Next.js provides routing and the production build, and Tailwind provides the light and dark themes.

The back end is developed using Python and FastAPI. A REST API connects the front end and the server, SQLAlchemy talks to the database, and PostgreSQL stores the data permanently.

Learner code is executed in the browser with Pyodide and Web Workers, and the server only checks the results.

The application can be installed on a phone or computer as a Progressive Web App while reusing the same web code.

Periodic Field Project Report

Name of the Student	Aryan Sawant
Program /Semester	III
Roll No/Seat No	[ROLL / SEAT NO e.g. BSCS/III-2627-XXXX]
Field project Title	Codeingo
Name of the Faculty mentor	Ms. Pranali Patil
Overview (Max 150 Words) (Brief summary of key activities, tasks and projects undertaken during the period)	
This chapter focused on identifying the problem, the requirements and the resources needed for developing Codeingo. The main problems identified were confusing syntax, lack of guidance, no real-world application and loss of motivation. The things in the system (course path, lessons, exercises, XP and streak) and the actions on them were specified. Hardware and software requirements were listed, and a conceptual model showing how a learner moves through the system was prepared.	
Learning Outcomes (Max 100 words): (Highlight the main lessons, skills, or knowledge gained during the week.)	
Learning outcomes from this chapter are: • Learned how to identify and define a software problem. • Understood the functional requirements of the Codeingo system. • Learned how to identify hardware and software requirements. • Understood the importance of system analysis before development. • Gained an understanding of the conceptual structure of the proposed system.	
Mentor Feedback and Suggestions (Max 100 words): (Feedback received from the mentor and any additional suggestions for improvement.)	
[Feedback from the guide, Ms. Pranali Patil, to be written after the chapter review]	

Student Sign with date

Mentor's Signature with date

Chapter 3 Requirements and Analysis

3.1. Problem Definition

The overall problem we are addressing is that many beginners start learning programming and then quit. Most tutorials are long, text-heavy and assume tools are already installed, and nobody tells the learner what to learn next. The survey confirmed this: of the learners who tried before, 13 gave up, and the biggest reasons for stopping were lack of guidance and no time. We can divide the problem into three sub-problems:

- The Guidance Problem: Learners do not know the order in which to learn topics and get lost between many tutorials.
- The Practice Problem: Reading or watching is not enough; learners need to write code and get instant feedback, but setting up a compiler is difficult.
- The Motivation Problem: Without rewards, a streak or friends, learners lose interest after a few days.

3.2. Requirement Specification

Independent of how we code the system, Codeingo needs to handle a specific set of core things and actions to be useful for a learner:

Things in the System:

- The Learning Path: Courses divided into units and lessons, unlocked one after another.
- The Exercise: A question (choice, fill-in-the-blank or a small program) that is graded by the server.
- The Progress Record: XP, daily streak, hearts and badges of each learner.
- The Test and Project: Chapter tests that unlock the next unit and a project at the end of each section.

Actions Performed on These Things:

- Learners can sign in with Google and complete 2-step verification.
- Learners can open a lesson, read the explanation and answer or run code exercises.
- Learners can take a chapter test, practise mistakes later and complete the daily challenge.
- Learners can follow friends, see leagues and enter a weekly contest.
- Administrators can manage content and users from a hidden admin area.

3.3. Software & Hardware Requirements

3.3.1. Hardware Requirements

To develop and run the application, the following minimal hardware configuration is required:

- Processor: Intel Core i3 or AMD Ryzen 3 (or equivalent mobile processor).
- RAM Capacity: Minimum 4 GB of system memory (8 GB recommended for development).

- Disk Capacity: At least 1 GB of free storage space for project code and packages.
- Input Devices: A standard mouse, trackpad, keyboard or a capacitive touch screen.
- Display: Any standard display that can render a responsive layout (phone to desktop).

3.3.2. Software Requirements

The development tools, libraries and environments used to write and package the application include:

- Operating System: Windows 10/11, macOS or Linux.
- Code Editor: Visual Studio Code as the primary editor.
- Runtime Environments: Node.js 20 or higher for the front end and Python 3.11 or higher for the server.
- Database: SQLite for development and PostgreSQL for production.
- Testing Environments: Modern web browsers (Chrome, Edge or Safari) and the pytest tool for back-end tests.

3.4 Conceptual Model

3.4.1 Conceptual Model

The diagram shows how a learner moves through Codeingo: sign-in with 2-step verification, the learning path, the choice of activity (lesson, test, practice, friends and contests), server-side grading, and rewards that end with a certificate.

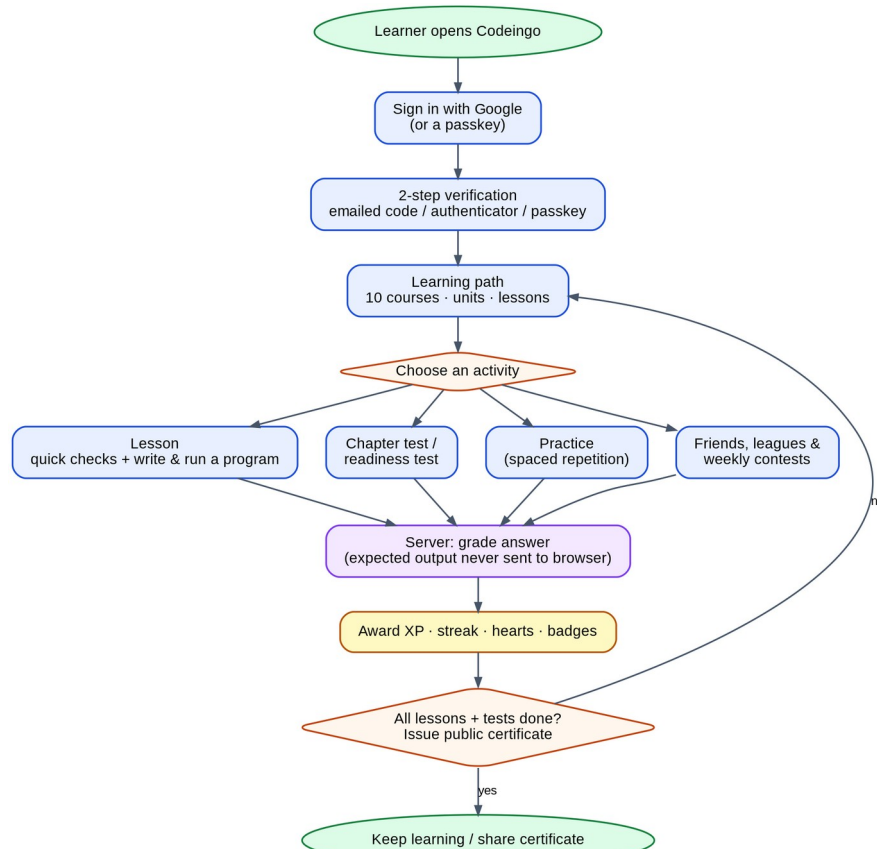


Fig. 3.4.1: Conceptual Model of Codeingo

Periodic Field Project Report

Name of the Student	Aryan Sawant
Program /Semester	III
Roll No/Seat No	[ROLL / SEAT NO e.g. BSCS/III-2627-XXXX]
Field project Title	Codeingo
Name of the Faculty mentor	Ms. Pranali Patil
Overview (Max 150 Words) (Brief summary of key activities, tasks and projects undertaken during the period)	
This chapter focused on collecting and analysing user responses through a survey conducted for the Codeingo project. The Coding Platform Interest Survey asked 53 learners about their age group, earlier programming experience, difficulties, preferred learning style, platforms used, interest in game-like learning, first language, motivation, solo or community preference, and reasons for quitting. The responses were interpreted to understand user requirements. The results showed strong interest in guided, hands-on and game-like learning with real projects and friendly competition.	
Learning Outcomes (Max 100 words): (Highlight the main lessons, skills, or knowledge gained during the week.)	
Learning outcomes from this chapter are: • Learned how to design a survey for collecting user requirements. • Learned how to analyse and interpret survey responses. • Understood how user feedback can influence software requirements. • Identified the important features preferred by potential users. • Learned how to present survey findings in an organised manner.	
Mentor Feedback and Suggestions (Max 100 words): (Feedback received from the mentor and any additional suggestions for improvement.)	
[Feedback from the guide, Ms. Pranali Patil, to be written after the chapter review]	

Student Sign with date

Mentor's Signature with date

CHAPTER 4 Data Analysis: Interpretation and Presentation

4.1 Survey Form

The survey was prepared as a Google Form titled “Coding Platform Interest Survey”. It contains the following questions:

1. What is your name?

2. Which age group are you in?

- Under 15
- 15–18
- 19–22
- 23+

3. Have you tried learning programming before?

- Yes, currently learning
- Yes, but gave up
- No, never tried

4. If you tried before, what made it difficult or boring?

- Confusing syntax
- Lack of guidance
- No real-world application
- Lost motivation
- Too much theory
- Other

5. How do you prefer to learn new skills?

- Watching videos
- Hands-on practice
- Playing games
- Interactive challenges
- Reading text

6. Have you ever used a coding platform before (like Scratch, Codecademy, freeCodeCamp, etc.)? Which one?

7. On a scale of 1–5, how appealing is the idea of learning to code through a game-like experience?

- 1 (not appealing)
- 2
- 3

- 4
- 5 (very appealing)

8. Which programming language would you be most interested in learning first?

- Python
- Java
- JavaScript
- C++
- Other
- Not sure

9. What would motivate you the most to keep learning on a platform like this?

- Building real projects
- Points/levels & rewards
- Competing with friends
- Certificates
- Just curiosity

10. Would you prefer learning solo or with a community (leaderboards, group challenges, forums)?

- Solo
- Community-based
- Mix of both

11. What's the biggest reason you'd stop using a learning platform like this?

- Not enough guidance
- No time
- Gets boring
- Lost interest
- Too difficult

4.2 Responses

The Google Form received 53 responses between 26 September 2026 and 3 October 2026.

The figure shows the first 30 responses (names and e-mail addresses are left out for privacy).

Timestamp	Age group	Has tried programming before	What made it difficult	How do you prefer to learn	Game-like appeal	First language
2026/09/26	15-18	Yes, but gave up	Confusing syntax	Hands-on practice	4	Python
2026/09/26	15-18	No, never tried	Other	Watching videos	5	Python
2026/09/26	19-22	Yes, currently learning	Too much theory	Interactive challenges	4	JavaScript
2026/09/26	15-18	No, never tried	No real-world application	Playing games	5	Not sure
2026/09/26	15-18	Yes, but gave up	Lack of guidance	Hands-on practice	3	Java
2026/09/26	19-22	Yes, currently learning	Lost motivation	Reading text	4	Python
2026/09/26	15-18	No, never tried	Other	Playing games	5	JavaScript
2026/09/26	15-18	Yes, but gave up	Confusing syntax	Watching videos	4	Python
2026/09/26	19-22	Yes, currently learning	Too much theory	Hands-on practice	3	C++
2026/09/26	15-18	No, never tried	No real-world application	Interactive challenges	5	Not sure
2026/09/26	15-18	Yes, but gave up	Lack of guidance	Playing games	4	JavaScript
2026/09/26	19-22	No, never tried	Other	Reading text	3	Python
2026/09/26	15-18	Yes, currently learning	Lost motivation	Hands-on practice	5	Java
2026/09/26	15-18	No, never tried	No real-world application	Watching videos	4	Not sure
2026/09/26	19-22	Yes, but gave up	Confusing syntax	Interactive challenges	4	Python
2026/09/26	19-22	Yes, currently learning	Lack of guidance	Watching videos	5	Python
2026/09/26	15-18	Yes, currently learning	Confusing syntax	Playing games	5	Java
2026/09/26	15-18	Yes, currently learning	-	Reading text	4	C++
2026/09/26	19-22	No, never tried	Other	Hands-on practice	1	Not sure
2026/09/26	19-22	Yes, currently learning	No real-world application	Watching videos	3	Java
2026/09/26	19-22	Yes, currently learning	Lack of guidance	Hands-on practice	5	Other
2026/09/26	19-22	Yes, currently learning	-	Watching videos	4	Python
2026/09/26	19-22	Yes, but gave up	Lack of guidance	Watching videos	2	C++
2026/09/26	15-18	Yes, currently learning	Lost motivation	Watching videos	4	Python
2026/09/26	15-18	Yes, currently learning	Lack of guidance	Hands-on practice	1	Python
2026/09/26	15-18	No, never tried	-	Watching videos	5	Not sure
2026/09/28	19-22	Yes, but gave up	Confusing syntax	Playing games	2	Java
2026/09/28	19-22	Yes, currently learning	Lack of guidance	Watching videos	3	Python
2026/09/28	Under 15	Yes, currently learning	Confusing syntax	Reading text	1	JavaScript
2026/09/28	15-18	Yes, currently learning	Confusing syntax	Hands-on practice	3	Not sure

Fig. 4.2.1: Responses

4.3 Data Analysis

Total responses: 53. Each question is shown as a chart followed by its interpretation.

Q2. Age group of the respondents

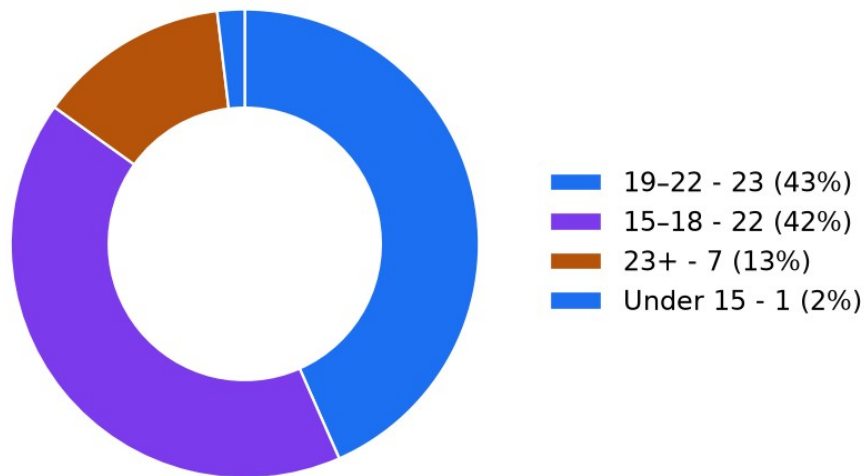


Fig. 4.3.1: Age group of the respondents (Question 2)

Interpretation: The respondents are young learners: 23 (43%) are aged 19–22 and 22 (42%) are aged 15–18, so 85% are school or college age. Seven (13%) are 23 or older and one is under 15. This matches the target users of Codeingo.

Q3. Have you tried learning programming before?

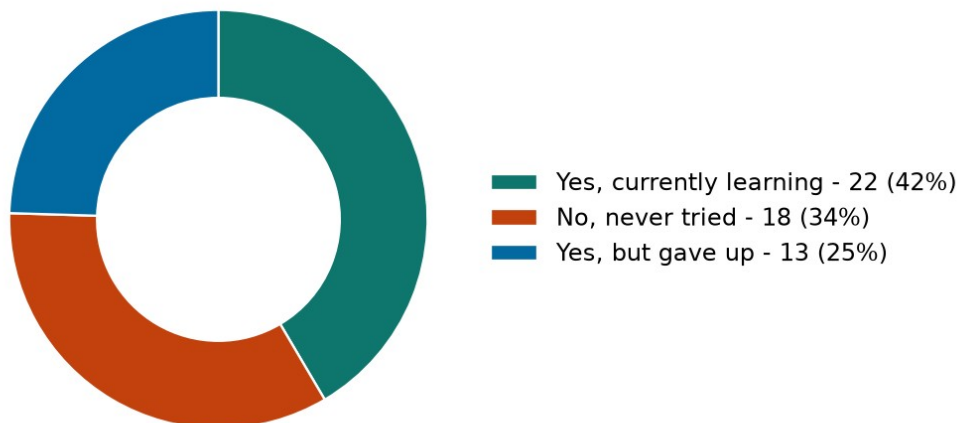


Fig. 4.3.2: Earlier programming experience (Question 3)

Interpretation: 22 respondents (42%) are currently learning, 18 (34%) have never tried and 13 (25%) tried but gave up. So about one in four beginners quits, and one in three has not even started – both groups need an easy, guided start.

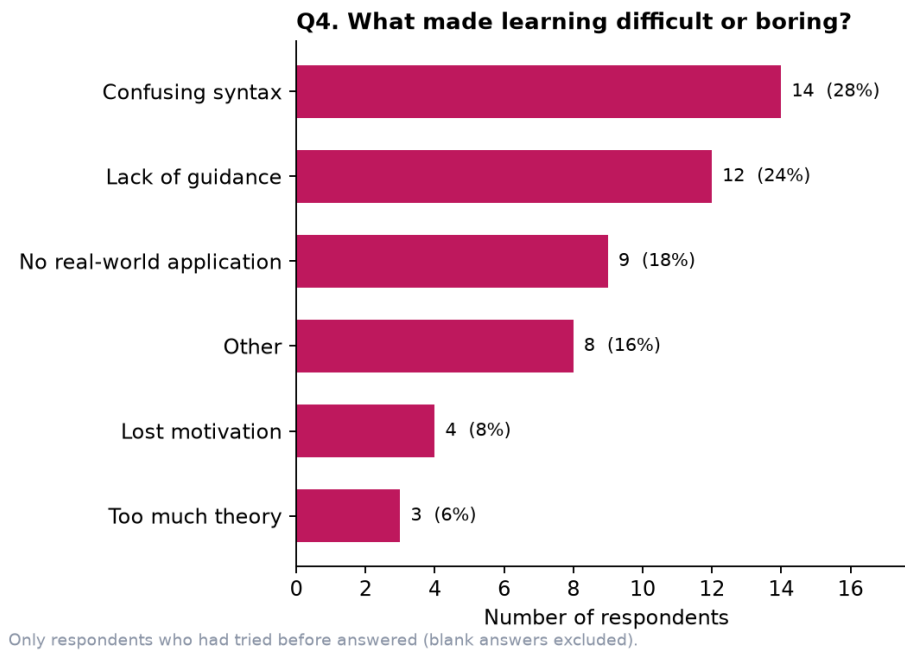


Fig. 4.3.3: What made learning difficult or boring (Question 4)

Interpretation: Confusing syntax (14) and lack of guidance (12) are the two biggest difficulties, followed by no real-world application (9). Together they account for 35 of the 50 answers. This supports short guided lessons, hints and projects in Codeingo.

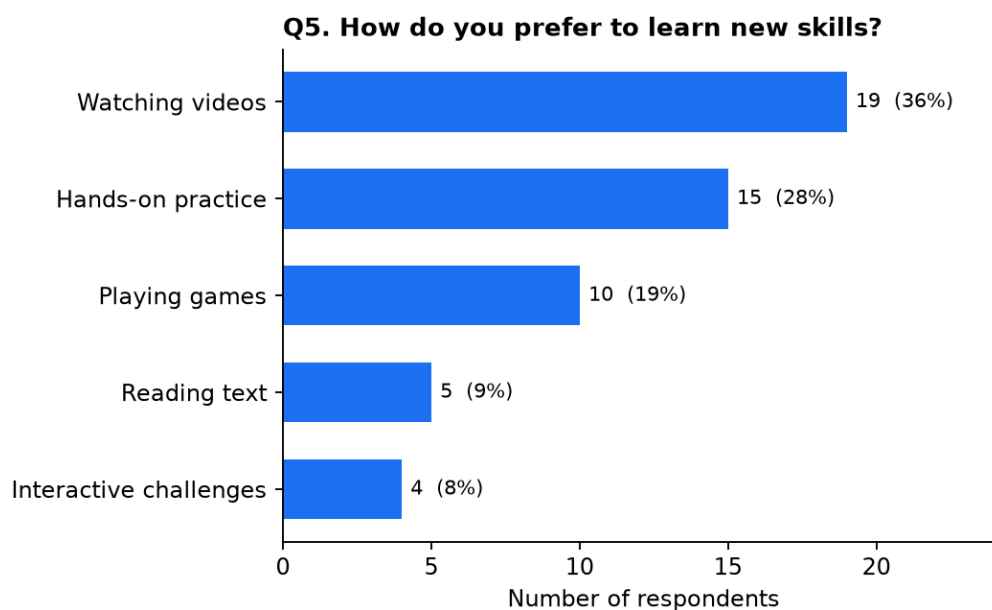


Fig. 4.3.4: Preferred way of learning (Question 5)

Interpretation: Watching videos is preferred by 19 (36%), hands-on practice by 15 (28%) and playing games by 10 (19%). Together hands-on practice, games and interactive challenges make up 29 answers (55%), which supports an interactive, practice-first design. Video lessons are noted as a future improvement.

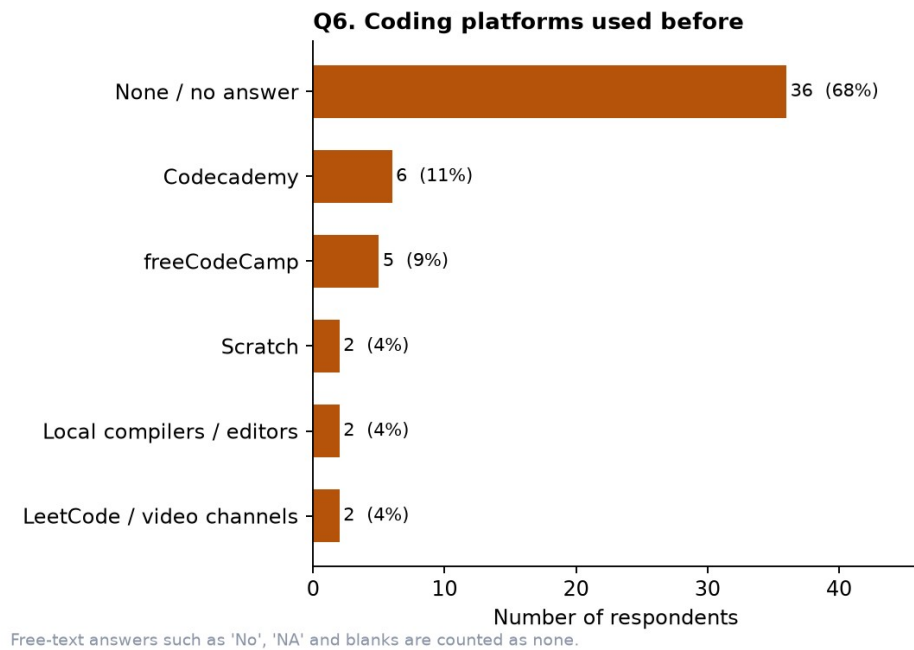


Fig. 4.3.5: Coding platforms used before (Question 6)

Interpretation: 36 of 53 respondents (68%) gave no platform at all, meaning they have not used a coding platform. Of the 17 who named one, Codecademy (6) and freeCodeCamp (5) are the most common. Many beginners therefore have no experience with such platforms, and a very simple start is important.

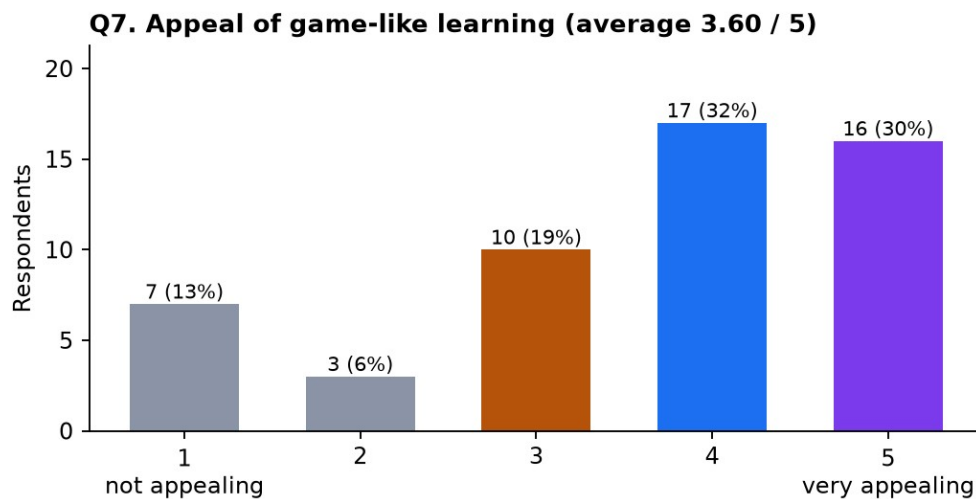


Fig. 4.3.6: Appeal of game-like learning (Question 7)

Interpretation: The average rating is 3.6 out of 5. 17 respondents gave 4 and 16 gave 5, so 33 (62%) rated it 4 or 5, while only 10 (19%) gave 1 or 2. This shows a clear interest in learning to code through a game-like experience.

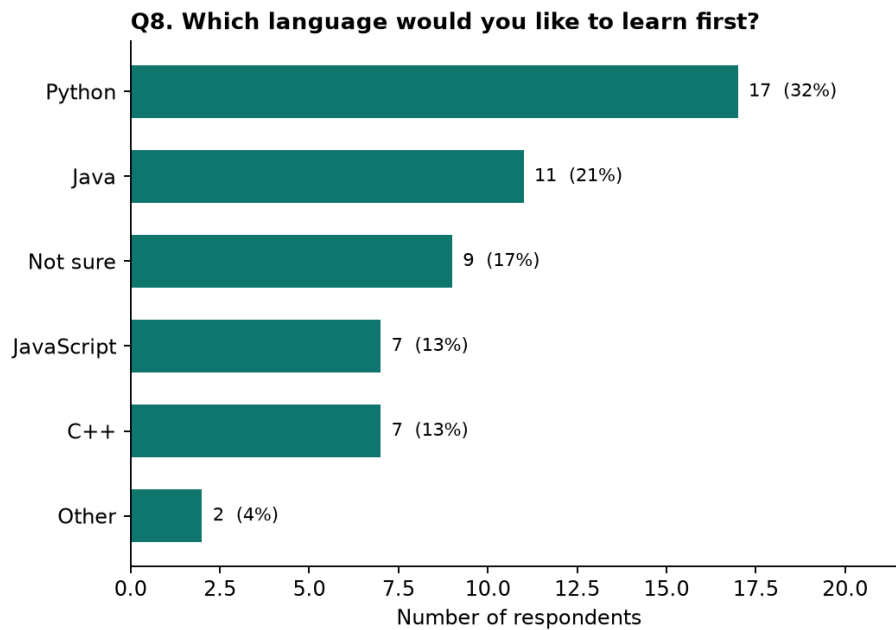


Fig. 4.3.7: First language to learn (Question 8)

Interpretation: Python is the most wanted first language (17, 32%), followed by Java (11, 21%) and JavaScript and C++ (7 each). Nine respondents (17%) are not sure. Codeingo therefore starts with Python and also offers Java, JavaScript and C++.

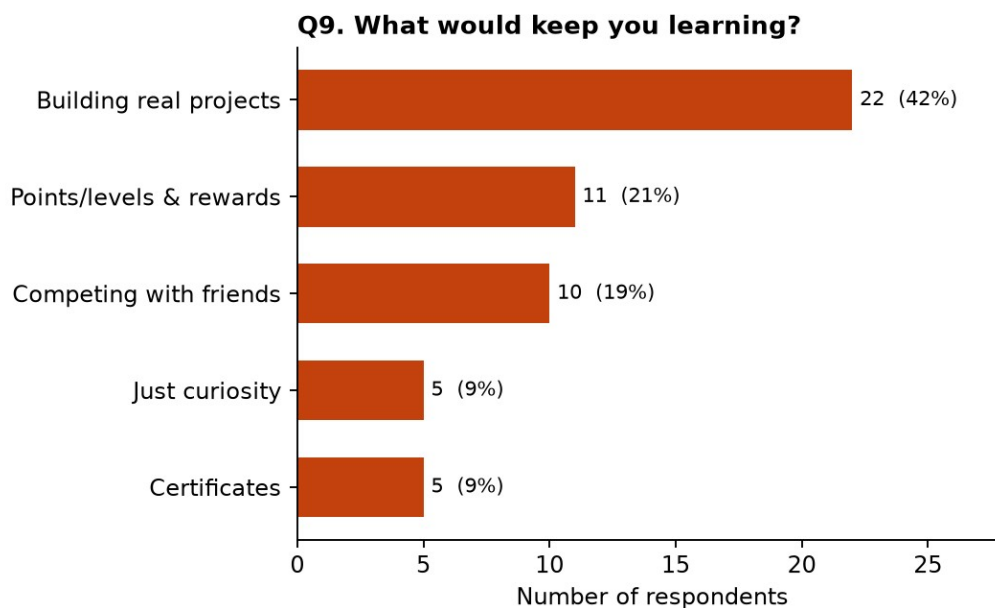


Fig. 4.3.8: What keeps learners motivated (Question 9)

Interpretation: Building real projects is the strongest motivator (22, 42%), followed by points/levels and rewards (11) and competing with friends (10). This is why Codeingo has projects at the end of every section, XP, badges and leagues.

Q10. Solo or community?

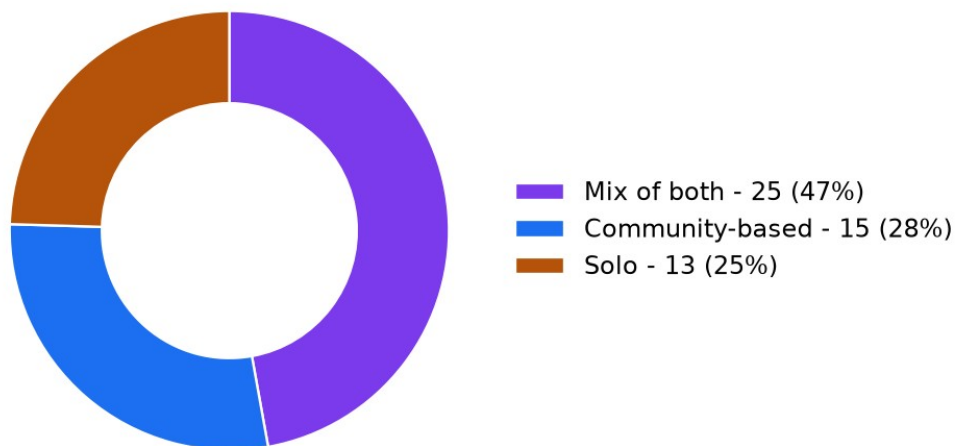


Fig. 4.3.9: Solo or community learning (Question 10)

Interpretation: 25 respondents (47%) prefer a mix of both and 15 (28%) prefer a community, while only 13 (25%) prefer to learn alone. So 75% want some community, which supports friends, leagues and contests.

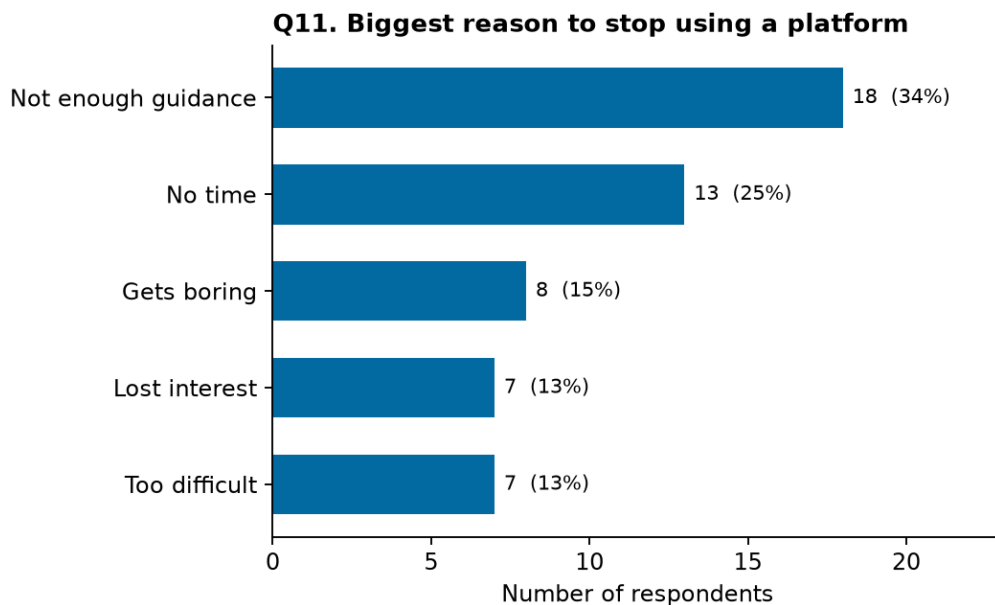


Fig. 4.3.10: Biggest reason to stop using a platform (Question 11)

Interpretation: Not enough guidance is the biggest reason to quit (18, 34%), followed by no time (13, 25%). Gets boring, lost interest and too difficult together account for 22 answers. Short five-minute lessons, clear next steps and daily streak reminders respond to these reasons.

4.4 Conclusion

The survey and analysis conducted for the Codeingo project show the need for a simple, guided and engaging platform for beginners. The responses highlight issues such as confusing syntax,

lack of guidance, no real-world application and loss of motivation. The survey also shows interest in game-like learning (62% rated it 4 or 5), hands-on practice, real projects, and learning together with friends.

The proposed Codeingo brings these ideas together in one application: a guided path of lessons, code that runs in the browser, XP, streaks, badges, projects, friends and contests. The survey also considers the preference for Python as a first language and for community learning, which shaped the course list and the social features.

Overall, the survey helped identify the practical requirements of Codeingo and provided a basis for developing a user-friendly platform that makes learning to program engaging and habit-forming.